

Adaptive Demand-Based Caching with Effective Address Filtering (ADC-EAF)

The paper 'The V-Way Cache : Demand-Based Associativity via Global Replacement' aims at reducing conflict misses and providing best victim elimination. This requires maintaining more number of tag stores as compared to a corresponding set-associative cache of the same size. These extra tag stores are then dynamically mapped to data stores from different sets that are identified as less reused by the global replacement policy. Thus the associativity of high demand sets increases at the expense of dormant low demand sets.

However, the V-way cache does not address cache pollution, allowing all blocks to occupy the cache. This would render the increased associativity inefficient as the additional ways will be filled with low reuse blocks. For this purpose we propose to combine the V way cache with the effective address filter (EAF)

Evicted blocks are tracked by the EAF. If missed blocks are accessed again, they are deemed as higher reuse and inserted into the cache with a higher priority. A higher priority means these do not fall under the scope of the replacement policy and will therefore not be evicted. As can be inferred, in a peculiar case all blocks may become high reuse blocks, requiring that the EAF be flushed. Further, the hardware cost of storing all evicted addresses is mitigated by using a bloom filter.

Implementation using Intel Pin:

We plan to first implement a fully functional V way cache followed by addition of the EAF.

What is needed for the V way cache:

- The hardware implementation of the V way cache uses forward pointers (FPTRs) and reverse pointers (RPTRs) for hits and misses both.
- With Pin's execution flow, we believe the FPTRs and RPTRs are needed to implement the global replacement policy only. Also these are slightly different. Both the FPTRs and RPTRs shall be an array of tags.
-
- If a set has room to grow, we use a global replacement policy that needs an array of saturating 2 bit counters to track reuse.
- Data block with 0 counter value is to be replaced, the corresponding tag victim can be found by RPTRs[index of counter with value 0] in the FPTRs array. This tag is to be invalidated and the invalid tag stores FPTRs array entry is changed.

What is needed for the EAF:

- We plan to first begin by using a large array to keep track of every address evicted. Later this shall be converted to a hash map in C++ to represent a bloom filter.

- To avoid high reuse blocks from being evicted, they must be marked in the cache. Marking is done by maintaining a `high_priority_ways` array. If an entry in this array is set, such a way is avoided by the reuse policy's loop.
- On a miss, search the EAF, if address is found, replace tag and set `high_priority_ways` array entry to 1. If all entries become 1, clear the array.

Testing and Validation

- Determine the ratio of the global replacement policy invocations over the number of cache misses for different workloads.
- Run simulations on Intel Pin across a range of benchmarks, especially those having wide disparity in set demand.
- Evaluate the impact of bloom filter size on misses by starting out with a large array and converging to a bloom filter.
- Analyze the impact on overall performance, instruction per cycle (IPC), and impact on cache-hit latency.

Design Choices and Optimizations

- Vary the Tag to Data Store ratio, to analyse the impact of providing maximum higher associativity.
- Use other replacement policies like PLRU, NMRU instead of LRU when no invalid tags are left
- Invalid tag-store entries can be used to maintain inclusion information without the need for duplicating cache lines in the data-store.
- EAF suffers from first access being deemed useless. To prevent this some sets avoid EAF.
- Bloom filter may give false positives. Need to vary bloom filter parameters to identify best possible hash function

References:

- [1] D. Sanchez and C. Kozyrakis, "The ZCache: Decoupling Ways and Associativity," *2010 43rd Annual IEEE/ACM International Symposium on Microarchitecture*, Atlanta, GA, USA, 2010, pp. 187-198. [The ZCache: Decoupling Ways and Associativity](#)
- [2] A. Jain and C. Lin, "Back to the Future: Leveraging Belady's Algorithm for Improved Cache Replacement," *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, Seoul, Korea (South), 2016, pp. 78-89. [Leveraging Belady's Algorithm for Improved Cache Replacement](#)
- [3] Vivek Seshadri, Onur Mutlu, Michael A Kozuch, Todd C Mowry. "The evicted-address filter: a unified mechanism to address both cache pollution and thrashing". PACT '12: Proceedings of the 21st international conference on Parallel architectures and compilation techniques;355 – 366. [Unified Approach to address Cache Thrashing and Pollution](#)